Ders 6 Çalışma Özeti

Ders 6 Çalışma Özeti

Genel Konular

- Command kalıbı: amaç ve GoF örneği (ekrandan, 00:10:00)
  - Amaç: nesnelere giden istekleri (mesajları) de nesne haline getirmek. Hoca bunu "metotları nesne yapmak" gibi düşünmeyi önerdi. Böylece istekler kuyruklanabilir, günlüğe (log) yazılabilir, undo/redo gibi işlemler yapılabilir.
  - GUI örneği: uygulamada menü, menüde MenuItem'lar vardır. Bir MenuItem'a tıklanınca onunla ilişkili Command nesnesinin Execute() metodu çalışır. Command soyut bir sınıftır; asıl iş, Execute()'u override eden somut alt sınıflarda yapılır.
  - Slayttaki OpenCommand: Execute() ve AskUser() metotları vardır, Application'a (`Add(Document)`) bir işaretçi tutar. Execute()'un sözde kodu: `name = AskUser(); doc = new Document(name); application->Add(doc); doc->Open()`.
  - Tasarım kararı: birçok somut komut Application'a işaretçi tutacaksa bu ilişki üst sınıf Command'a taşınabilir. PasteCommand gibi belgeye bağlı komutlarda Document ilişkisi ise yalnız o alt sınıfta kalabilir.
  - Menü içinde alt menüler de istenirse Composite ile Command birlikte kullanılmalıdır; hoca bunu alıştırma olarak bıraktı.
- Command kalıp yapısı (ekrandan, 00:12:20)
  - Bileşenler: Client, Invoker (komutu veren), soyut Command (Execute()), ConcreteCommand (Execute(), state), Receiver (Action(); komutun üstünde yürütüleceği varlık).
  - ConcreteCommand'ın `receiver` ilişkisi vardır; Execute() içinde `receiver->Action();` çağrılır. Client'tan ConcreteCommand'a kesikli çizgi, komut nesnesini Client'ın oluşturduğunu gösterir; Client ayrıca komutun alıcısını (Receiver) da belirler.
  - Command'ın Java anlamında katı bir interface olması gerekmez; alt sınıfların çoğu bir receiver kullanıyorsa command-receiver ilişkisi üst sınıfa taşınabilir.
- Command örnek gerçeklemesi: akıllı ev kumandası (ekrandan, 00:20:00, 00:21:00)
  - Kaynak: Head First Design Patterns. Merkezi bir kontrol paneline aydınlatma, elektronik alet ve iklimlendirme düğmeleri bağlanır.
  - Paket `dp.command.example`: `AcKapa` arayüzü (`boolean ac()`, `boolean kapa()`); `AydinlatmaDugmesi implements AcKapa` (isim alanı, `toString()` "AcKapa:" + isim döndürür, ac/kapa iskelet olarak `true` döner).
  - `Kumanda`: `ArrayList<AcKapa> acKapaDugmeleri; dugmeEkle, ac(int dugmeNo), kapa(int dugmeNo), hepsiniKapa()`. Ana programda "sokak kapısı açılınca antre ışığını ve televizyonu aç, hava soğuksa kombiyi aç" gibi senaryolar tanımlanır.
- Iterator kalıbı: amaç ve iki çözüm (00:22:47, ekrandan, 00:30:00)
  - Amaç: parçalardan oluşan bir bütünün (aggregate) parçalarını, iç yapısını açığa çıkarmadan istenen biçimde dolaşabilmek. Akla ilk gelen bütün bir veri yapısıdır (dizi, liste, ağaç…).
  - Sorun: farklı dolaşım biçimleri (baştan sona, geriye doğru, dairesel listede bir tur) ve farklı eleman türleri için veri yapısının arayüzünü şişirmek istemiyoruz.
  - Çözüm 1: List ve onunla ilişkili ListIterator (First(), Next(), IsDone(), CurrentItem()). Veri yapısı ile dolaşma biçimi ayrılır; ama her veri yapısına bir iterator gerekir ve istemci somut türlere bağlanır.
  - Çözüm 2 (slayt 28): soyut AbstractList (CreateIterator(), Count(), Append(Item), Remove(Item)) ve Iterator arayüzü; List–ListIterator, SkipList–SkipListIterator çiftleri. Client yalnız soyut türlerle çalışır.

  - Hoca bu yapının Bridge'e benzediğini söyledi: farklı kalıpların sınıf şemaları çok benzeyebilir, onları ayıran amaçları, çalışma biçimleri ve gerçekleme ayrıntılarıdır.
- Iterator kalıp yapısı ve gerçekleme ayrıntıları (00:31:33)
  - Bileşenler: Aggregate, ConcreteAggregate, Iterator, ConcreteIterator; hepsi soyut sınıf ya da arayüz üzerinden kullanılır.
  - Iterator dolaştığı bileşenleri değiştirebilmeli mi? Değiştirmesin isteniyorsa aggregate kendi klonunu verir; izin verilecekse elemanlar referansla verilir.
  - Dolaşım algoritmasını kim gerçekleştirmeli? Sorumluluk, gerekli bilgiye en çok sahip olana verilir; bu da çoğunlukla iterator'dır. Iterator elemanları tek tek de verebilir, biriktirip toplu da verebilir (örneğin ters sırada).
- Iterator örnek gerçeklemesi: iki restoranın menüsü (ekrandan, 00:40:00, 00:43:40, 00:48:00, 00:50:00, 00:50:40)
  - Kaynak: Head First Design Patterns. Kahvaltıda uzman Pancake House ve ana yemekte uzman Diner firmaları satın alınmış; menüleri tek menüde, ayrıca vejetaryen, yalnız kahvaltı ya da yalnız ana yemek listesi olarak sunulmak isteniyor.
  - Ortak `MenuItem` (ad, açıklama, vejetaryen mi, fiyat). `PancakeHouseMenu` elemanları `ArrayList<MenuItem>` içinde tutar; `DinerMenu` dizi (`MenuItem[]`) kullanır. İkisinin `getMenuItems()` metodu farklı tür döndürür.
  - Kalıpsız çözüm (`dp.iterator.exampleWithoutPattern`, `WaitressQuickAndDirty`): her menü için ayrı for döngüsü; `printMenu`, `printVegetarianMenu`, `printBreakfastMenu` gibi metotlarda kopyala-yapıştır (duplicate code) çoktur.
  - Kalıplı çözüm (`dp.iterator.exampleWithPattern`): `Menu` arayüzü (`createIterator(): Iterator<MenuItem>`), `DinerMenuIterator` ve `PancakeHouseMenuIterator` (`Iterator<MenuItem>`: hasNext(), next()). Firmaların menü sınıflarına yalnız `createIterator()` eklenir.
  - `Waitress.printMenu()` iki menüden iterator alır ve tek bir `printMenu(Iterator<MenuItem>)` metoduyla dolaşır. Yeni menüler (tatlı, farklı mutfaklar) eklendikçe kalıplı çözümün avantajı artar.
- Memento kalıbı (ekrandan, 01:10:00, 01:12:10)
  - Amaç: nesnenin durum bilgisini sarmalama (encapsulation) ilkesini bozmadan elde etmek. Böylece nesne ikincil saklama ortamına yazılabilir, geri yüklenebilir, ağ üzerinden aktarılabilir, undo düzeneği kurulabilir. Durumun yalnız bir kısmını saklamak da yeterli olabilir.
  - Yapı: Originator (SetMemento(Memento m), CreateMemento(), state), Memento (GetState(), SetState(), state), Caretaker (memento listesini tutar). `CreateMemento(): return new Memento(state);` `SetMemento(m): state = m->GetState()`.
  - Hocanın benzetmesi: çocuğun her yıl çekilen fotoğrafları. Çocuk Originator, fotoğraflar Memento, albümü saklayan Caretaker'dır; fotoğrafların dışarıdan değiştirilmediğinden emin olmak gerekir.
  - Sorun: Memento'nun durumunu yalnız Originator değiştirebilmeli. C++'ta Memento'nun metotları private yapılıp Originator `friend` ilan edilir. Java'da Memento ile Originator aynı pakete konup paket mühürlenebilir (sealed) ya da Memento, Originator'ın iç sınıfı yapılır.
  - Java'da `createMemento()` durumu doğrudan değil klonunu vermelidir; yoksa nesne referansı paylaşılır ve geri dönme çalışmaz (hoca bunu kendi denemesinde yaşadığını anlattı).
- Memento örnek gerçeklemesi: iç sınıfla çözüm (ekrandan, 01:20:00, 01:30:00)
  - Paket `dp.mementoV2.innerClass`: `State` (Cloneable, String içerik), `Originator` (modify, createMemento, setMemento), Originator'ın public iç sınıfı `Memento` (private getState/setState).
  - `Caretaker.test()`: "ilk durum" → memento al → "yeni durum" → `setMemento(mem)` → nesne ilk duruma döner. Dışarıdan memento'nun durumu değiştirilemez.
  - `testBroken()`: sahte bir Originator'dan alınan memento gerçek nesneye verilirse engellenemiyor. Slayttaki çözüm önerisi: memento'ların idaresini Originator'ın kendisi yapsın (caretaker `activateMemento(int siraNo)` çağırsın) ya da memento'ya Originator'ın hashCode'u ya da rastgele seri numarası konup uyumsuzlar reddedilsin.
- Observer kalıbı (01:30:07, ekrandan, 01:33:50)
  - Amaç: bir nesnenin durumu değişince ona bağlı bütün nesnelerin haberdar olması ve güncel bilgiye erişmesi; bunu sınıf adlarına doğrudan bağlaşım kurmadan, çalışma anında yapmak. MVC'nin altyapısına benzer (hesap tablosu verisi değişince tablo, çubuk grafik ve pasta grafik güncellenir).

  - Yapı: Subject (Attach(Observer), Detach(Observer), Notify(): `for all o in observers { o->Update() }`), ConcreteSubject (GetState(), SetState(), subjectState), Observer (Update()), ConcreteObserver (Update(): `observerState = subject->GetState()`).
  - Subject ile Observer arasında bire-çok ilişki vardır. Bir observer birden fazla subject'i de gözleyebilir.
- Observer gerçekleme ayrıntıları (ekrandan, 01:40:00)
  - Pull model: gözlenen yalnız "değiştim" der, gözleyiciler ilgili durumu kendileri sorgular. Bağlaşım azalır, gözleyici yalnız istediği kısmı alır; ama neyin değiştiğini kendisi çıkarmak zorundadır.
  - Push model: gözlenen, durum değişikliği hakkında ayrıntılı bilgi (belki bütün durumunu) gönderir. Subject'e daha fazla kod yazılır.
  - IoT gibi iletişimin pahalı olduğu ortamlarda pull/push seçimi önemlidir. Observer'ın sürekli "değiştin mi" diye sorması (polling) hem iletişim hem işlemci masrafıdır.
  - Haber vermeyi kim tetiklemeli? İstemci tetiklerse unutabilir; subject tetiklerse çok küçük değişiklikler de bildirilir (hocanın örneği: hassas bir saksı nem sensörü).
  - Kullanım: bir soyutlamanın birbirine bağlı birden fazla parçası olduğunda (örneğin bir composite'te bir bileşenin değişmesinin diğerlerine yayılması) ya da bağlaşımı artırmadan haber vermek gerektiğinde. Java GUI olay mimarisi (listener'lar) bu kalıba dayanır.
- Observer örnek gerçeklemesi: müşteri kaydı (ekrandan, 01:50:00, 01:52:40)
  - Çözüm 1: `Customers.addCustomer` hem hoş geldiniz mektubunu hem adres doğrulamayı doğrudan çağırır.
  - Yeni gereksinim: müşteri bir şubenin 10 km yakınındaysa mektuba indirim kuponu eklensin; benzer gereksinimler de gelecek. Çözüm 2: `Customer` (static `Vector<ICObserver> myObs`, static `attach/detach`, `getState()`, `notifyObs()`), `ICObserver` arayüzü (`update(myCust: Customer)`), gözleyiciler `AddrVerification`, `BrickAndMortar`, `WelcomeLetter`.
  - Brick-and-mortar: ağırlıklı e-ticaret yapanların fiziksel mağazalarda yaptıkları ticarete verdikleri ad. Kupon dağıtımı durdurulacaksa BrickAndMortar gözleyicisi detach edilir, yeniden başlayınca attach edilir.
  - Yalnız müşteri nesneleri izlenecekse soyut bir Subject üst sınıfına gerek yoktur.
- Strategy kalıbı (02:04:48, ekrandan, 02:10:00)
  - Amaç: aynı işi yapan ama farklı avantaj ve dezavantajları olan algoritmaları, bu işi isteyen istemcilerden soyutlamak; böylece algoritma çalışma anında değiştirilebilir.
  - GoF örneği: metin düzenleyicide satır bölümleme (line break). Hoca Word'de iki yana yaslama ve hece bölme ayarlarıyla gösterdi. `Uygulama` (`- bölmeleyici: SatırBölümleme`, `+ metinDüzenle()` → `bölmeleyici.bölmele()`), soyut `SatırBölümleme` (`bölmele() {abstract}`), alt sınıflar `KesmeliBöl`, `KelimedenBöl`, `SınırdanBöl`.
  - Yapı: Context, Strategy (arayüz ya da soyut sınıf), ConcreteStrategy'ler. Context'te `setStrategy` vardır. Strateji context'in verisine ihtiyaç duyarsa veri parametreyle verilir ya da stratejiye context referansı konur.
  - Zayıf yön: istemciler farklı stratejilerin varlığından haberdar olmak zorundadır; seçimi üçüncü bir nesne yaparsa bu yük istemciden alınır. Dağıtık ortamda strateji ile context arasındaki iletişim masrafı da düşünülmelidir.
- Strategy ve Builder karşılaştırması (02:15:48)
  - Sınıf şemaları neredeyse aynıdır. Builder'da yönetici (director) parçaları nasıl oluşturup birleştireceğini bilir ve inşayı adım adım yönetir. Strategy'de context işin nasıl yürüdüğünü bilmez; bütün sorumluluğu stratejiye verir.
- Strategy örnek gerçeklemesi: uluslararası vergi hesabı (ekrandan, 02:20:00, 02:22:30)
  - Kaynak: Design Patterns Explained. Çözüm 1: `enum Country { TR, US, EU }` ve `TaskController.taxAmount(double income)` içinde switch-case (TR: 0.18, EU: 0.15, US: 0.12, varsayılan: 0.10).
  - Gerçek vergi hesabında her case yüzlerce satır olabilir. if-else zinciri ya da switch-case yazdığınızı görürseniz kalıtım ve çok biçimlilikle daha iyi bir tasarım yapılabilir demektir.
  - Çözüm 2: `TaxRule` arayüzü (`taxAmount(income: double): double`), `TRrules`, `EUrules`, `USrules`; `TaskController`'da `tax: TaxRule` ve `setTax(tax: TaxRule)`.
  - Farklı vergi türleri (gelir, araç, emlak) birbirinden çok farklıysa her biri için ayrı strateji hiyerarşisi kurulabilir (`IncomeTaxRule`, `VehicleTaxRule`).
  - Hocaya göre her somut strateji Singleton yapılabilir, çünkü bir ülke için tek bir kural

takımı vardır.
- State kalıbı (02:26:31, ekrandan, 02:30:00)
  - Amaç: nesnenin iç durumu değiştiğinde davranışının da (büyük ölçekte) değişmesini sağlamak.
  - GoF örneği: TCP bağlantısı belli bir anda Established, Listen ya da Closed durumundadır ve her durumda Open/Close/Acknowledge farklı davranır. Switch-case yerine "neyin değiştiğini bul ve sarmala" ilkesiyle TCPConnection'a soyut TCPState türünden bir `state` üyesi konur; istek `state->Open()` gibi bu üyeye iletilir. Alt sınıflar TCPEstablished, TCPListen, TCPClosed.
  - Strategy'den farkı: somut durum sınıfları context'i başka bir duruma geçirebilir; bu yüzden state'ten context'e de bir ilişki vardır. Durum geçişleri sık, strateji değişimleri seyrektir. Sınıf yapısı ise aynıdır.
- State örnek gerçeklemesi: kişisel bilgi yöneticisi (ekrandan, 02:34:45)
  - Kayıtlar temiz (clean) ya da kirli (dirty) olabilir. Kirliyken çıkmak istenince kaydedilmemiş değişiklik için onay sorulur; temizken doğrudan çıkılır.
  - Paket `dp.state.example`: soyut `State` (`save(File f, Serializable s)` ObjectOutputStream ile yazar; `edit()`), alt sınıflar `CleanState` ve `DirtyState`. Temiz durumdayken edit gelince durum kirliye çekilir; kirliyken save gelince kaydedilip temize çekilir.

Hocanın Özellikle Vurguladığı Kısımlar

- Benzer sınıf şemaları, aynı kalıp demek değildir
  - Iterator'ın Bridge'e, Strategy'nin Builder'a ve State'e benzemesi örneklerinde hoca, kalıpları amaçlarının, çalışma biçimlerinin ve gerçekleme ayrıntılarının ayırdığını söyledi. Birbirine benzeyen iki kalıbın farklarının sorulabileceğini de belirtti (00:29:36, 02:17:49).
- GoF gösterimi yerine standart UML
  - Dörtlü çetenin gösterimi UML'den önce çıkmıştır. Observer şemasındaki bire-çok ilişki eski gösterimde siyah nokta ile çizilmiştir; hoca kalıtım ve arayüz gerçekleme oklarının standart UML'e göre çizilmesini beklediğini hatırlattı (01:33:12).
- Sorumluluk, bilgiye en çok sahip olana verilir
  - Iterator'da dolaşım algoritmasının yeri bu ilkeyle seçilir: çoğunlukla iterator (00:33:28).
- Switch-case uyarısı
  - if-else zinciri ya da switch-case yazıyorsanız kalıtım ve çok biçimlilikle daha iyi bir tasarım yapılabilir (02:21:11).
- "Neyin değiştiğini bul ve sarmala"
  - Strategy'de satır bölümleme algoritmaları, State'te TCP durumları bu ilkeyle ayrı sınıflara alınır (02:09:09, 02:28:05).
- Observer'da durum bilgisinin gönderilmesi
  - Pull ve push modellerini ve avantaj-dezavantajlarını iyi bilmek gerekir; birinin avantajı ötekinin dezavantajıdır (01:36:48).
- KISS ilkesi
  - "Keep it simple, stupid": Java'nın kalıbı daha basit gerçekleyen hazır yapıları işe yarıyorsa kullanılmalıdır (00:25:54).

Kısa Tekrar Notları

- Command: istek = nesne. Invoker → Command.Execute() → ConcreteCommand → receiver.Action(). Kuyruklama, loglama, undo/redo sağlar.
- Iterator: Aggregate.createIterator() ile somut iterator alınır; istemci hasNext()/next() ile veri yapısını bilmeden dolaşır.
- Kalıpsız menü çözümünde her veri yapısı için ayrı döngü ve duplicate code vardır; kalıplı çözümde tek `printMenu(Iterator)` kalır.
- Memento: Originator.createMemento() / setMemento(m); Caretaker yalnız saklar. Java'da iç sınıf + private metotlar + klonlanmış durum.
- Observer: attach/detach/notify; observer.update(). Pull: "değiştim" + sorgu; push: ayrıntılı bilgi gönderilir.

- Strategy: context + setStrategy + çalışma anında değişen algoritma; istemci strateji türlerini bilmek zorunda kalabilir.
- Strategy ve Builder şeması aynıdır; Builder'da yönetici adımları yönetir, Strategy'de iş bütün olarak stratejiye verilir.
- State: context'in state üyesi; somut durumlar context'in durumunu değiştirebilir; geçişler sıktır.

Detaylı Açıklamalar

- Command kalıbında istemci, menü öğesinin hangi nesnenin hangi metodunu çağıracağını önceden sabitlemek zorunda kalmaz. Menü öğesi (Invoker) yalnız bir Command tutar ve Execute() çağırır. OpenCommand örneğinde Execute(), AskUser() ile dosya adını alır, yeni bir Document oluşturur, Application'a ekler ve açar. Hoca, PasteCommand gibi belgeye bağlı komutlarda Document işaretçisinin alt sınıfta kalabileceğini, bütün komutların ihtiyaç duyduğu Application işaretçisinin ise üst sınıfa taşınabileceğini anlattı. Kalıp yapısında Client hem ConcreteCommand'ı oluşturur hem de Receiver'ını belirler. Akıllı ev örneğinde Kumanda nesnesi `AcKapa` düğmelerini bir listede tutar ve numarasıyla düğmeyi açar ya da kapatır.

  Ders sonrası not: Slayttaki `Kumanda.ac(int dugmeNo)` metodu, düğme yoksa `acKapaDugmeleri.get(dugmeNo) == null` sınamasıyla false dönmeyi amaçlıyor. Oysa Java'da `ArrayList.get` geçersiz bir sırada null döndürmez, `IndexOutOfBoundsException` fırlatır (küçük bir Java programıyla denendi). Sınamanın `dugmeNo < 0 || dugmeNo >= acKapaDugmeleri.size()` biçiminde olması gerekir.
- Iterator kalıbında kalıpsız çözümün iki zayıflığı vardır: istemci (garson) her menünün iç veri yapısını bilmek zorundadır ve aynı dolaşma kodu birçok metotta kopyalanır. Kalıplı çözümde firmaların menü sınıflarına yalnız `createIterator()` eklenir (hoca bunu Eclipse'in "Compare With" özelliğiyle gösterdi). `DinerMenuIterator` diziyi bir `position` sayacıyla dolaşır; `hasNext()` konum dizi sonuna geldiyse ya da eleman null ise false döner. `PancakeHouseMenuIterator` ise ArrayList'in kendi metotlarından yararlanır. `remove()` sonradan eklenebilecek bir iş olarak bırakılmıştır. Vejetaryen menüde kalan küçük tekrar da `printMenu`'ya ikinci bir "yalnız vejetaryen" parametresi eklenerek giderilebilir. İki menüde sınıf sayısı artmış görünse de menü sayısı arttıkça kalıplı çözümün getirdiği ek sınıf sayısı farkı göreli olarak azalır.
- Memento kalıbında istenen, durumun bir bütün olarak, üstünde oynanmadan saklanmasıdır. Nesnenin kendisi saklanırsa public setter'lar üzerinden durum değişebilir. Hocanın iç sınıf çözümünde `Memento`, `Originator`'ın public iç sınıfıdır ama bütün metotları private'tır; Originator onlara erişir, Caretaker erişemez. Hoca, durumu klonlamadan döndürdüğünde nesne referanslarının paylaşıldığını ve `setMemento` sonrası nesnenin "yeni durum"da kaldığını, sorunu `State`'i Cloneable yapıp klon döndürerek çözdüğünü anlattı. Sahte bir Originator'dan alınan memento'nun gerçek nesneye verilmesi hâlâ engellenemiyor. Hoca, ağır olmayan bir çözüm bulan öğrencilerin önerilerini beklediğini söyledi.
- Observer kalıbında Subject'in gözleyicileri soyut Observer türünden tutması, gözlenenin gözleyicilerin somut türlerini bilmemesini sağlar. Belirli bir ConcreteObserver belirli bir ConcreteSubject'e erişmek zorundaysa alt düzeyde bir ilişki kurulur; böyle bir zorunluluk yoksa ilişki üst sınıflara taşınabilir. Belirli bir subject yalnız belirli türden gözleyicileri kabul edecekse `attach` içinde `instanceof` ile tür sınaması yapılabilir. Müşteri örneğinde her gözleyici kendi işine `update()` ile başlar; var olan sınıfların metot adlarını değiştirmeye gerek yoktur, yalnız update metodu eklenip asıl işi yapan metodu çağırması sağlanır.
- Strategy kalıbında context ile stratejiler arasındaki veri alışverişi bir tasarım kararıdır. Stratejilerin çoğu context'e ihtiyaç duyuyorsa ilişki üst sınıfa (o zaman strateji arayüz değil soyut sınıf olur) konabilir; azı ihtiyaç duyuyorsa yalnız onlara verilir. İkinci yol kod karmaşıklığını biraz artırır ama iletişim masrafını düşürür.
- State kalıbında TCPListen durumundayken close gelirse bağlantı Closed durumuna geçer; Established durumundayken close gelirse (hocanın örneğinde) bir şey yapılmaz ve belki false dönülür. Geçişleri context de yapabilir, somut durumlar da. Kişisel bilgi yöneticisi örneğinde hoca, temiz durumdayken save sonrası durumu yeniden temize çekmenin gereksiz olduğunu fark edip kaynak kodda düzelteceğini söyledi (02:35:33).

  Ders sonrası not: Hoca, Dörtlü Çete'nin her somut stratejinin Singleton olabile-

ceğini söylediğini aktardı (02:25:07). GoF kitabında büyük olasılıkla Strategy için “strateji nesneleri iyi birer Flyweight olur”, State için ise “State nesneleri çoğunlukla Singleton’dır” denir. Bir ülkenin vergi kuralı nesnesini Singleton yapmak yine de geçerli bir tasarım seçeneğidir.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.